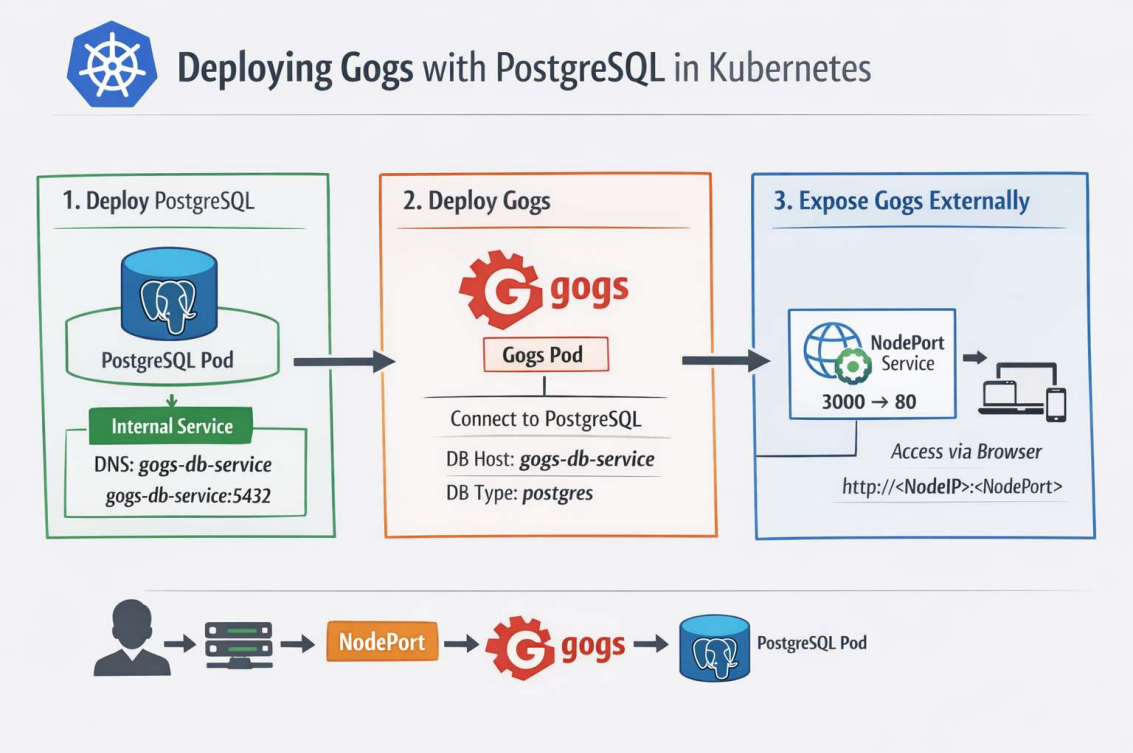
Deploying Gogs with PostgreSQL in Kubernetes

This task involves setting up a Gogs application with a PostgreSQL database inside a Kubernetes cluster. First, PostgreSQL is deployed as a pod and configured with database credentials and an initial database name. A Kubernetes Service is created to expose PostgreSQL internally, allowing other applications in the cluster to connect to it using a service DNS name.



Next, the Gogs application is deployed and configured to use PostgreSQL as its backend by specifying the database type as "postgres" and providing the database service name as the host. To make Gogs accessible to users, a NodePort Service is created, which exposes the application outside the cluster through a web browser. Finally, the setup is verified by checking that all pods and services are running properly. This task demonstrates how Kubernetes manages application deployment, networking, and service communication within a cluster.

Create the PostgreSQL deployment

```
controlplane:~$ kubectl create deployment gogs-db --image=postgres:14
deployment.apps/gogs-db created
controlplane:~$
```

Configure database credentials and initial database name

```
controlplane:~$ kubectl set env deployment/gogs-db POSTGRES_PASSWORD=shiva-pass POSTGRES_DB=gogs_db
deployment.apps/gogs-db env updated
controlplane:~$
```

Expose PostgreSQL internally

This creates the DNS name 'gogs-db-service' for the frontend to use

```
controlplane:~$ kubectl expose deployment gogs-db --port=5432 --name=gogs-db-service
service/gogs-db-service exposed
controlplane:~$
```

Verify Service

```
controlplane:~$ kubectl get svc gogs-db-service
NAME              TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)    AGE
gogs-db-service   ClusterIP   10.96.126.88   <none>        5432/TCP   9m43s
controlplane:~$
```

Deploy the Frontend (Gogs)

Gogs will connect to the PostgreSQL service. Note that Gogs listens on port **3000** by default for the web interface.

Create the Gogs deployment

```
controlplane:~$ kubectl create deployment gogs-frontend --image=gogs/gogs
deployment.apps/gogs-frontend created
controlplane:~$
```

Configure Gogs to use the PostgreSQL backend

We tell Gogs the DB type is 'postgres' and point it to our service name

```
controlplane:~$ kubectl set env deployment/gogs-frontend DB_TYPE=postgres DB_HOST=gogs-db-service:5432 DB_NAME=gogs_db DB_USER=postgres DB_PASSWD
=shiva-pass
deployment.apps/gogs-frontend env updated
controlplane:~$
```

Expose Gogs to external traffic

Gogs internal port is 3000, we map it to 80 on the service for easy access

```
controlplane:~$ kubectl expose deployment gogs-frontend --port=80 --target-port=3000 --type=NodePort --name=gogs-access-service
service/gogs-access-service exposed
controlplane:~$
```

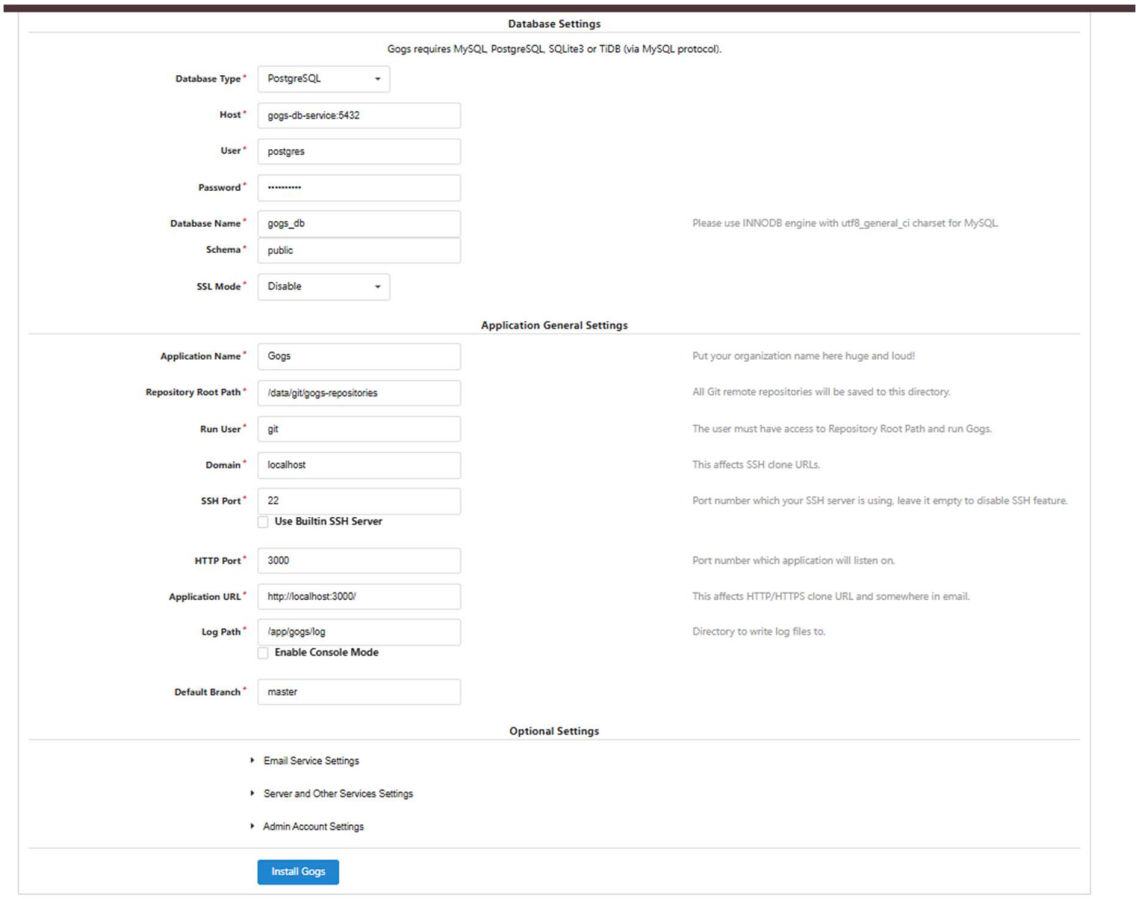
Check if pods are 'Running'

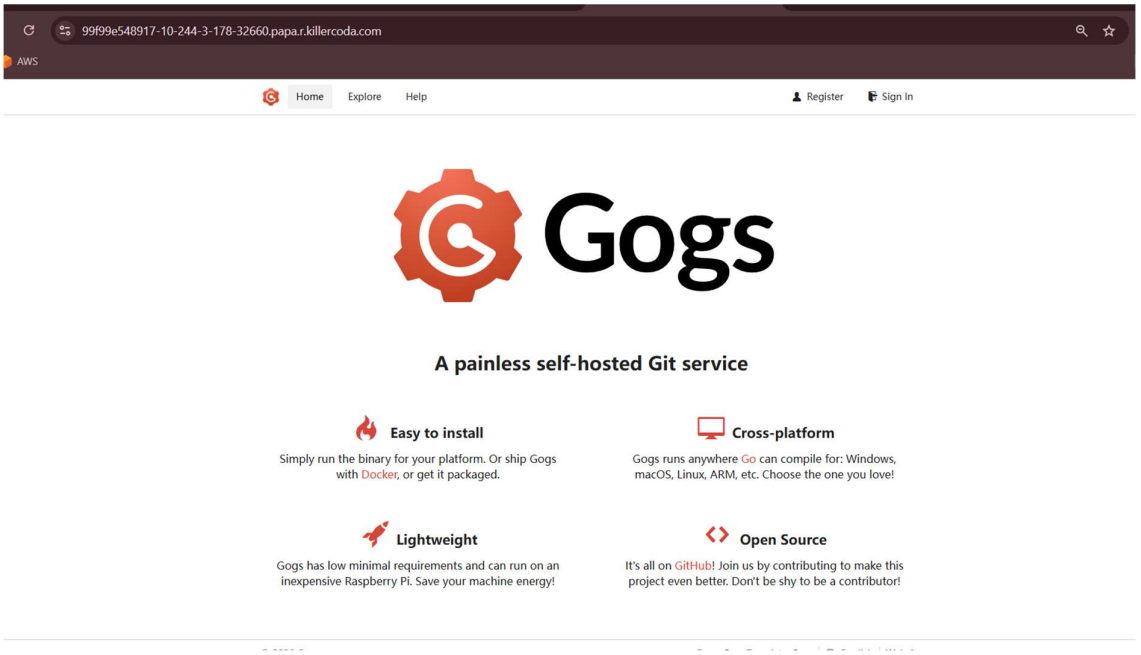
```
controlplane:~$ kubectl get pods
NAME                             READY   STATUS    RESTARTS   AGE
gogs-db-545776f586-nqcfn         1/1     Running   0          7m33s
gogs-frontend-75f8f454cb-k7sjn   1/1     Running   0          58s
controlplane:~$
```

Find the NodePort to access Gogs in your browser

```
controlplane:~$ kubectl get svc gogs-access-service
NAME                 TYPE       CLUSTER-IP       EXTERNAL-IP   PORT(S)        AGE
gogs-access-service   NodePort   10.107.226.163   <none>        80:30330/TCP   68s
controlplane:~$
```

Verify


Database Settings
Gogs requires MySQL, PostgreSQL, SQLite3 or TiDB (via MySQL protocol).
Database Type * PostgreSQL
Host * gogs-db-service:5432
User * postgres
Password * ••••••••••
Database Name * gogs_db
Please use INNODB engine with utf8_general_ci charset for MySQL.
Schema * public
SSL Mode * Disable
Application General Settings
Application Name * Gogs
Put your organization name here huge and loud!
Repository Root Path * /data/git/gogs-repositories
All Git remote repositories will be saved to this directory.
Run User * git
The user must have access to Repository Root Path and run Gogs.
Domain * localhost
This affects SSH clone URLs.
SSH Port * 22
Port number which your SSH server is using, leave it empty to disable SSH feature.
Use Builtin SSH Server
HTTP Port * 3000
Port number which application will listen on.
Application URL * http://localhost:3000/
This affects HTTP/HTTPS clone URL and somewhere in email.
Log Path * /app/gogs/log
Directory to write log files to.
Enable Console Mode
Default Branch * master
Optional Settings
Email Service Settings
Server and Other Services Settings
Admin Account Settings
Install Gogs


99f99e548917-10-244-3-178-32660.papa.r.killercoda.com
AWS
Home
Explore
Help
Register
Sign In
Gogs
A painless self-hosted Git service
Easy to install
Simply run the binary for your platform. Or ship Gogs with Docker, or get it packaged.
Cross-platform
Gogs runs anywhere Go can compile for: Windows, macOS, Linux, ARM, etc. Choose the one you love!
Lightweight
Gogs has low minimal requirements and can run on an inexpensive Raspberry Pi. Save your machine energy!
Open Source
It's all on GitHub! Join us by contributing to make this project even better. Don't be shy to be a contributor!